

---

```

int badApple(int[][] M, int n)
{
    boolean infected = true
    int count = 0
    while infected do
        infected = false
        for i = 1 to n do
            for j = 1 to n do
                if M[i][j] == -1 then
                    infected = infected or infect(M, i - 1, j, n)
                    infected = infected or infect(M, i + 1, j, n)
                    infected = infected or infect(M, i, j - 1, n)
                    infected = infected or infect(M, i, j + 1, n)
            count = count + 1
    return count - 1
}

```

---



---

```

int infect(int[][] M, int i, int j, int n)
{
    if 1 ≤ i ≤ n and 1 ≤ j ≤ n then
        if M[i][j] == -1 then
            M[i][j] = -1
            return true
        else
            return false
    }
}

```

---

Si consideri una matrice composta da tutti 1, tranne il nodo in alto a sinistra (1,1):

|    |   |   |   |   |
|----|---|---|---|---|
| -1 | 1 | 1 | 1 | 1 |
| 1  | 1 | 1 | 1 | 1 |
| 1  | 1 | 1 | 1 | 1 |
| 1  | 1 | 1 | 1 | 1 |
| 1  | 1 | 1 | 1 | 1 |

Nell'esecuzione dei cicli **for** annidati, quando  $i = 1$  e  $j = 1$  il nodo (1,1) infetta i nodi (2,1) e (1,2); quando  $i = 1$  e  $j = 2$  il nodo (1,2) infetta i nodi (2,2) e (1,3); e così via, riempiendo tutte le righe e tutte le colonne. Al completamento dei cicli **for** annidati, tutti i nodi vengono infettati e l'algoritmo restituisce 1.

Molti studenti hanno invece scelto la strada di una visita in profondità. Questo approccio è sbagliato, perché una visita in profondità non calcola la lunghezza del percorso minimo fra due nodi, ma potenzialmente può identificare percorsi molto lunghi.

Nella matrice seguente, per esempio, partendo dal nodo in posizione (1,1), una visita in profondità potrebbe percorrere l'intero bordo prima scendendo, poi andando a destra, poi salendo, poi andando a sinistra. La lunghezza di tale percorso è  $4n - 4$ , mentre la distanza fra l'angolo alto-sinistra e l'angolo basso-destra è pari a  $2n - 2$ . L'algoritmo restituirebbe quindi un valore sbagliato.

|    |   |   |   |   |
|----|---|---|---|---|
| -1 | 1 | 1 | 1 | 1 |
| 1  | 0 | 0 | 0 | 1 |
| 1  | 0 | 0 | 0 | 1 |
| 1  | 0 | 0 | 0 | 1 |
| 1  | 1 | 1 | 1 | 1 |

## Esercizio B1 – Math ways (2) – Punti $\geq 8$

Il problema può essere risolto tramite backtracking. Si crea un vettore  $S$  contenente i valori della sequenza, inizializzando il primo elemento a  $k$ . Dopo di che, si chiamerà la procedura ricorsiva che prova ad aggiungere gli elementi successivi, utilizzando una delle tre operazioni previste. Se il valore precedente nella sequenza è

maggiore di  $n$ , si farà backtrack senza stampare nulla; se si è raggiunto  $n$ , viene stampata la sequenza e si termina; altrimenti si fa un ulteriore passo di backtrack.

Una stima molto grezza del costo computazionale è  $O(n \cdot 3^n)$ , perché ad ogni chiamata ricorsiva è possibile fare tre diverse scelte e il costo di stampa è  $O(n)$ . Tuttavia, il numero di sequenze è molto più basso, grazie al fatto che la moltiplicazione e l'elevamento al quadrato fanno crescere velocemente i valori.

---

```
mathways(int n, int k)
```

---

```
  int[] S = new int[1...n]
  mwRec(S, 1, k, n)
```

---



---

```
mwRec(int[] S, int i, int n)
```

---

```
  S[i] = k
  if S[i] == n then
    | print S[1...i]
  else if S[i] < n then
    | mwRec(S, i + 1, S[i] + 1, n)
    | mwRec(S, i + 1, S[i] * 2, n)
    | mwRec(S, i + 1, (S[i])2, n)
```

---

Si noti che il valore iniziale  $k = 3$  è stato scelto per evitare il caso particolare in cui  $k = 2$  dà origine allo stesso valore quando si calcola  $2k$  e  $k^2$ . L'avevo inserito nel compito di Febbraio 22, l'avevo tolto nel compito di Giugno 22 perché mi ero completamente dimenticato, ma fortunatamente Cristian se ne è accorto.

## Esercizio B2 – Piano strategico – Punti $\geq 10$

Il problema può essere risolto tramite una rete di flusso.

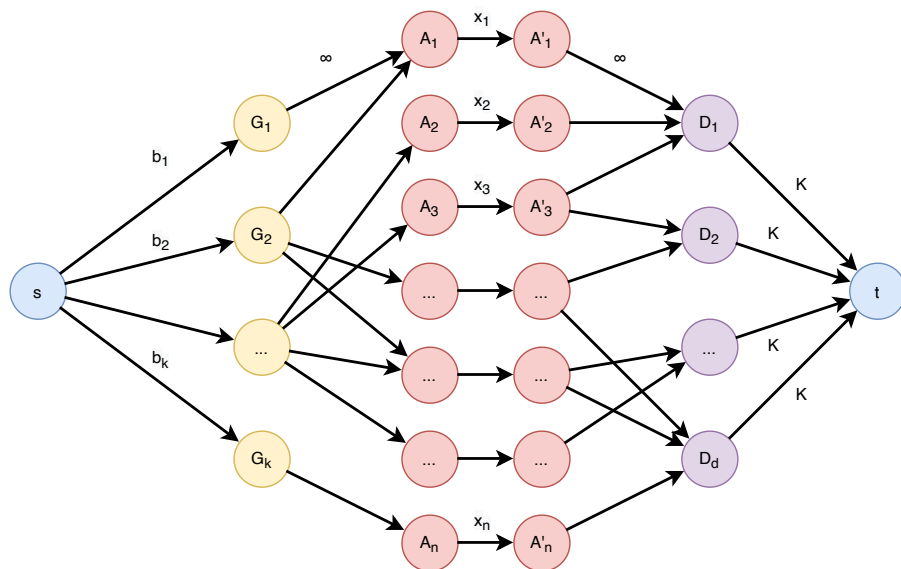
Costruiamo una rete  $G = (V, E, s, t, c)$ . L'insieme  $V$  dei nodi è costituito nel modo seguente:

- un nodo  $s$  (sorgente);
- un nodo  $t$  (pozzo);
- un nodo  $G_j$  per ognuno dei gruppi;
- una coppia di nodi  $A_i$  e  $A'_i$  per ognuna delle azioni;
- un nodo  $D_l$  per ognuno dei delegati

L'insieme degli archi è costituito nel modo seguente:

- un arco  $(s, G_j)$  per ogni gruppo  $g_j$ , con capacità  $b_j$  (il budget del gruppo); in questo modo ogni gruppo riceverà al più  $b_j$  budget;
- un arco  $(G_j, A_i)$  per ogni gruppo  $j \in grp_i$ , che associa un'azione a un gruppo; possiamo utilizzare  $+\infty$  per la capacità, oppure  $b_j$ ; in entrambi i casi, il valore sarà limitato da  $b_j$ ;
- un arco  $(A_i, A'_i)$  per ogni azione, con capacità  $x_i$ ; questo serve ad assicurarsi che l'azione riceverà esattamente un budget  $x_i$ ;
- un arco  $(A'_i, D_l)$  per ogni delegato  $l \in del_i$ , che associa un'azione a un delegato; possiamo utilizzare  $+\infty$  per la capacità, oppure  $K$ ; in entrambi i casi, il valore sarà limitato da  $K$ ;
- infine, un arco  $(D_l, t)$  da ogni delegato al pozzo, con capacità  $K$ .

Il grafo risultante è il seguente:



Eseguendo uno degli algoritmi visti a lezione su questa rete, si otterrà una divisione che rispetta tutti i vincoli se e solo se ognuno degli archi  $(A_i, A'_i)$  avrà esattamente valore  $x_i$ .  
Per il calcolo della complessità, è possibile osservare che:

$$|V| = 2 + k + 2n + d = O(k + n + d)$$

$$|E| = k + O(kn) + O(dn) + d = O((k + d) \cdot n)$$

Il valore del flusso massimo atteso è pari a  $X = \sum_{i=1}^n x_i$   
Utilizzando il limite di Ford-Fulkerson, si ottiene  $O((k + d) \cdot n \cdot X)$ .

### Esercizio B3 – Double sequence – Punti $\geq 12$

Esistono diversi metodi per risolvere il problema tramite programmazione dinamica.

Sia  $DP[i][j]$  il numero di sequenze lunghe  $i$ , composte da numeri tutti minori di  $j$  e che rispettano i vincoli sul raddoppio. Questo valore può essere calcolato nel modo seguente:

$$DP[i][j] = \begin{cases} 0 & j = 0 \\ j & j > 0 \wedge i = 1 \\ DP[i-1][\lfloor j/2 \rfloor] + DP[i][j-1] & j > 0 \wedge i > 1 \end{cases}$$

- Se  $j = 0$ , non è possibile comporre sequenze di interi positivi.
- Se  $j > 0$  e  $i = 1$ , è possibile comporre una sequenza di un elemento composto da tutti i valori  $1, 2, \dots, j$  (che sono  $j$ ).
- Altrimenti, ci sono due possibilità:
  - si termina la sequenza con il valore  $j$  e quindi si considerano tutte le sequenze di lunghezza  $i - 1$  e valore massimo  $\lfloor j/2 \rfloor$ ;
  - si scarta il valore  $j$  e si considerano tutte le sequenze di lunghezza  $i$  con valore massimo  $j - 1$ .

---

```
int countSequences(int n, int m)
```

---

```
int[][] DP = new int[1...n][1...m] = {-1}
```

```
return csRec(DP, n, m)
```

---

---

```

int csRec(int[][] DP, int i, int j)


---


if j == 0 then
|   return 0
else if i == 1 then
|   return j
else
|   if DP[i][j] < 0 then
|   |   DP[i][j] = csRec(DP, i - 1, ⌊j/2⌋) + csRec(DP, i, j - 1)
|   return DP[i][j]

```

---

La complessità è pari a  $O(mn)$ , il costo per inizializzare e riempire la tabella.

Una soluzione alternativa, più costosa, è la seguente. Sia  $DP[i][j]$  il numero di sequenze composte da  $i$  elementi, che terminano esattamente con il valore  $j$  e che rispettano i vincoli sul raddoppio.  $DP[n][m]$  può essere calcolato nel modo seguente:

$$DP[i][j] = \begin{cases} 0 & j = 0 \\ 1 & j > 0 \wedge i = 1 \\ \sum_{k=1}^{\lfloor j/2 \rfloor} DP[i-1][k] & j > 0 \wedge i > 1 \end{cases}$$

- Se  $j = 0$ , non è possibile comporre sequenze di interi positivi.
- Se  $j > 0$  e  $i = 1$ , è possibile comporre una sequenza di un elemento composto dal solo valore  $j$ .
- Altrimenti, si termina la sequenza con il valore  $m$  e si sommano insieme tutte le sequenze di  $i - 1$  elementi che terminano con valori compresi fra 1 e  $\lfloor m/2 \rfloor$ , che quindi rispettano la regola del raddoppio.

Una volta ottenuta la matrice  $DP$ , il risultato finale è data dalla somma dell' $n$ -esima riga della matrice, ovvero la somma di tutte le sequenze che terminano con un valore compreso fra 1 e  $m$ :

$$\sum_{k=1}^m DP[n][k]$$

Una piccola ottimizzazione consiste nel notare che il valore iniziale della sommatoria può essere limitato superiormente da  $\lfloor \frac{m}{2^{n-1}} \rfloor$ , in quanto valori più grandi daranno origine a sequenze che contengono valori superiori a  $m$  (per via degli  $n - 1$  raddoppi).

Il codice per risolvere il problema è il seguente:

---

```

int countSequences(int n, int m)


---


int[][] DP = new int[1...n][1...m] = {-1}
int ret = 0
for k = 1 to ⌊ $\frac{m}{2^{n-1}}$ ⌋ do
|   ret = ret + csRec(DP, n, k)
return ret

```

---

```

int csRec(int[][] DP, int n, int m)


---


if m == 0 then
|   return 0
else if n == 1 then
|   return 1
else
|   if DP[n][m] < 0 then
|   |   DP[n][m] = 0
|   |   for i = 1 to ⌊m/2⌋ do
|   |   |   DP[n][m] = DP[n][m] + csRec(DP, n - 1, i)
|   return DP[n][m]

```

---

La complessità è pari a  $O(nm^2)$ , il costo per inizializzare e riempire la tabella, tenendo conto che riempire ogni casella costa  $O(m)$ .